AutoKart RC - Actuator Control Compact implementation report

PRIMARY IMPLEMENTATION: autokart_rc_v5_controlled_stop.py

This report summarizes how the workspace controller commands the steering, brake, and traction actuators. The v5 file is treated as the primary integrated design; full_bldc_test.py is documented as a separate direct-drive bench test.

1. System at a glance

```
Browser UI  ->  ESP32-S3 HTTP server/state machine  ->  actuator interfaces
                      |              |
                      |              +-- TB6600 step/dir: steering + brake
                      +-------------------- GP8630N I2C -> BLD750 SV (throttle)
```

Steering actuator Stepper motor through a TB6600. GPIO4 produces STEP pulses, GPIO5 selects DIR, and GPIO6 controls ENA. Positive motion is configured as clockwise/right.

Brake actuator Second TB6600 axis: GPIO10 STEP, GPIO11 DIR, GPIO12 ENA. Positive motion is clockwise brake application. v5 range: 0 to +15 degrees.

Throttle / traction actuator The ESP32 writes a 16-bit value to GP8630N over I2C. The module is in 0-10 V mode and its OUT feeds BLD750 SV; v5 software ceiling is 3.0 V.

2. Wiring and signal map

Function	ESP32-S3	Destination / meaning
Steering STEP	GPIO4	TB6600 PUL+
Steering DIR	GPIO5	TB6600 DIR+; positive = right
Steering ENA	GPIO6	TB6600 ENA+; active-low in code
Brake STEP/DIR/ENA	GPIO10/11/12	Second TB6600 axis
I2C	GPIO8 SDA, GPIO9 SCL	GP8630N, address 0x58
Throttle analog	GP8630N OUT	BLD750 SV; GP GND -> BLD COM

Supporting wiring references are rc_connection.txt, Steering Brake connections.txt, and throttle _details.

3. Steering and brake control

Each axis is a StepperAxis object. A browser command sets a target or nudge; the ESP32 clamps the angle, converts degrees to pulses, and the main loop emits one pulse at a time until the software target is reached.

v5 limits: steering -90 to +90 deg brake 0 to +15 deg
Calibration: pulses/degree = (3200 / 360) * gear ratio; ratios are 1.0 now
Timing: steering 1200 -> 500 us brake 1500 -> 700 us
Commands: left, right, nudge, hold, center

The position is OPEN LOOP: it is only a pulse count from an assumed physical zero. There is no encoder or limit-switch feedback. Physical zero and gear ratios therefore must be calibrated before relying on the angle estimate.

Actuator Control - Operation and Safety

4. Throttle control sequence

1. The UI sends a throttle request. The server clamps it to 0-3.0 V and accepts it only while armed and while the brake is released.
2. The request becomes throttle_target. Every 15 ms, throttle_update ramps the actual output toward the target.
3. v5 rises by 0.02 V/update and normally falls by 0.10 V/update. The DAC code is int((V / 10) * 65535), written to GP8630N register 0x02.
4. GP8630N OUT controls BLD750 SV.

BENCH-ONLY WARNING The source notes that the BLD750 previously faulted near 0.75 V. The 3.0 V ceiling is not electrically validated. Keep wheels off the ground until the fault and command scaling are resolved.

5. Brake, arm, stop, and watchdog logic

Boot: throttle 0 V; steering center; brake applied; disarmed.
Arm: throttle 0; steering holds; brake releases; commands accepted.
Brake on: throttle zero immediately; brake target = +15 deg clockwise.
Brake off: brake target = 0; throttle forced zero; new throttle is required.
Stop: disarm; ramp throttle down at 0.04 V every 15 ms; then brake.
Timeout: no heartbeat for 1.8 s while armed -> disarm, throttle zero, steering hold, brake apply.
E-stop: latched; throttle zero, steering hold, brake apply; commands block until reset. Reset leaves the system disarmed.

6. Standalone BLD750 actuator test

full_bldc_test.py separately drives BLD750 digital inputs through PN2222A transistors. A GPIO high turns the transistor on and pulls the input low.
GPIO13 -> EN: high/pulled low = RUN; low/open = STOP
GPIO14 -> BRK: high/pulled low = brake released; low/open = quick brake
GPIO15 -> F/R: direction selection (software labels CW and CCW)
Its safe stop sets GP8630N to 0 V, applies the BLD750 brake, and stops enable. Direction reversal is allowed only in the full safe state.

7. Practical validation checklist

- Confirm TB6600 microstep switches match 3200 pulses/revolution.
- Measure travel; update gear ratios and software limits.
- Verify DIR polarity and brake apply direction at low speed.
- Measure GP8630N output and verify 0-10 V scaling before traction testing.
- Test arm, brake/throttle interlock, controlled stop, E-stop, and 1.8 s timeout with wheels off the ground.
- Add physical end stops or feedback before treating open-loop angle as safety- critical position.

Source basis: autokart_rc_v5_controlled_stop.py (primary), rc_bench.py (earlier bench controller), full_bldc_test.py (standalone BLD750 test), and wiring notes.